

Programação de Sistemas

Programação com procedimentos e funções em C

Marcelo de Gomensoro Malheiros



Recapitulando...

» Repetição com while



- C:

- `int c = 1;`
- `while (c <= 5) {`
- `printf("%i\n", c);`
- `c++;`
- `}`

- Python:

- `c = 1`
- `while c <= 5:`
- `print(c)`
- `c = c + 1`

Recapitulando...

» Repetição com for



- C:

- `for (int c = 1; c <= 5; c++) {`
- `printf("%i\n", c);`
- `}`

- Python:

- `for c in range(1, 6):`
- `print(c)`

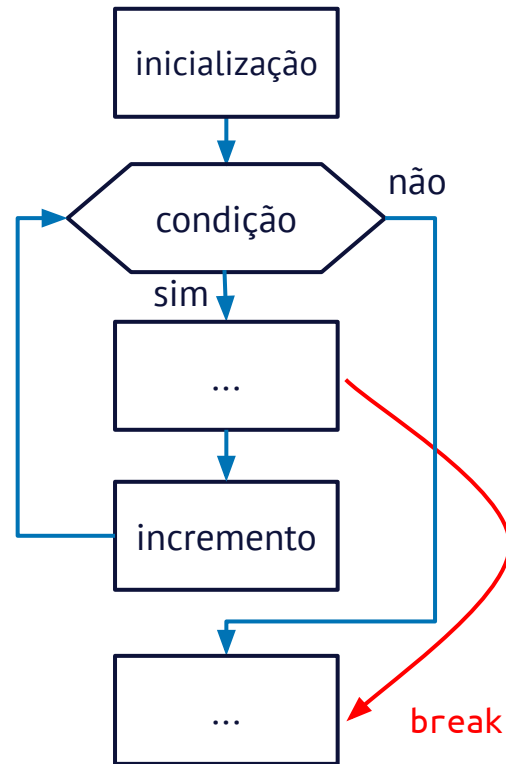
Recapitulando...

» Comando break



- Podemos interromper a repetição dentro do laço com o comando **break**:

```
○ for (int c = 1; c <= 5; c++) {  
○     if (c == 3) {  
○         break;  
○     }  
○     printf("%i\n", c);  
○ }
```



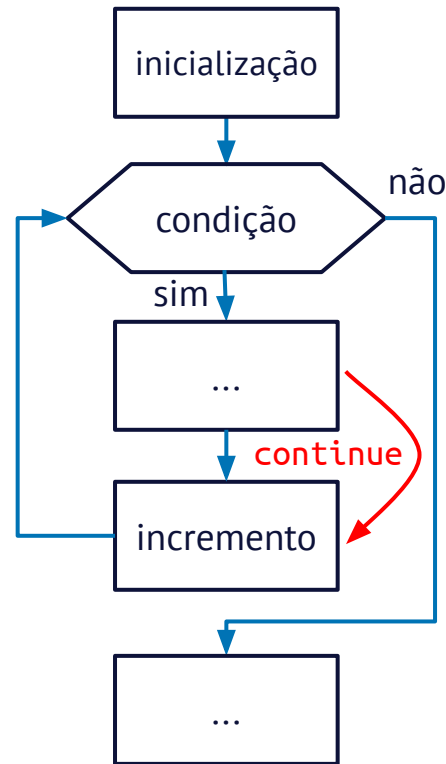
Recapitulando...

» Comando continue



- Podemos ir direto para a próxima repetição dentro do laço com o comando **continue**:

```
○ for (int c = 1; c <= 5; c++) {  
○     if (c == 3) {  
○         continue;  
○     }  
○     printf("%i\n", c);  
○ }
```



Procedimientos

- Um **procedimento** é um trecho de código com um **nome**
 - como não retorna nada, precisa ter **void** no início
 - **void hello() {**
 - **printf("Olá, mundo!\n");**
 - **}**
- É executado apenas quando chamado:
 - **int main() {**
 - **printf("iniciando...\n");**
 - **hello();**
 - **return 0;**
 - **}**

- Um procedimento pode ter **parâmetros e variáveis locais**:

- `void contagem(int n) {`
- `for (int c = 1; c <= n; c++) {`
- `printf("%i\n", c);`
- `}`
- `}`
- `int main() {`
- `contagem(5);`
- `return 0;`
- `}`

- Uma **função** é um procedimento com um **valor de retorno**:
 - `int ler_inteiro() {`
 - `int i = 0;`
 - `printf("Entre com um inteiro: ");`
 - `scanf("%i", &i);`
 - `return i;`
 - `}`
- O valor retornado por uma chamada precisa ser guardado
 - o tipo de retorno tem que ser o mesmo da variável que o recebe
 - `int n = ler_inteiro();`

- Uma função pode ter um ou mais parâmetros
 - É recomendável que o **return** seja sempre o último comando
 - `float maior(float x, float y) {`
 - `float m = x;`
 - `if (y > m) {`
 - `m = y;`
 - `}`
 - `return m;`
 - `}`

- Parâmetros e valor de retorno podem ser de qualquer tipo
 - Por enquanto, usaremos apenas `int`, `float` e `char`
 - `char escolha(char c1, char c2) {`
 - `char c = ' ';`
 - `while (c < c1 || c2 < c) {`
 - `printf("Escolha uma letra entre %c e %c: ", c1, c2);`
 - `scanf("%c", &c);`
 - `}`
 - `return c; // retorna um char`
 - `}`

Funções úteis

- `#include <ctype.h>`
- Teste de caracteres:
 - `char c = 'x';`
 - `isalpha(c) → 1` (verdadeiro)
 - `isdigit(c) → 0` (falso)
 - `isalnum(c) → 1` (verdadeiro)
 - `isspace(c) → 0` (falso)
- Conversão de letras:
 - `tolower(c) → 'x'`
 - `toupper(c) → 'X'`

- Compile com `gcc -Wall prog.c -o prog -lm`
- `#include <math.h>`
 - `float f = 1.23;`
 - `sqrt(f) / fabs(f)`
 - `exp(f) / log(f)`
 - `sin(f) / cos(f) / tan(f)`
 - `ceil(f) / floor(f) / round(f)`
 - `fmod(f, 1.0)`
 - `pow(f, 3.0)`
 - `M_E → 2.718282`
 - `M_PI → 3.141593`

Exercícios

- Faça um procedimento chamado `quadrado()`, que recebe um parâmetro inteiro `n` com o lado do quadrado a ser impresso, composto por caracteres `'#'`. Teste este procedimento.
- Faça uma função que calcula e devolve o *desvio padrão populacional* de três `float` passados como parâmetro.
- Faça uma função que recebe um `char` contendo uma letra e retorna a posição dela no alfabeto (de 1 a 26) ou 0 se não for uma letra.

Perguntas?